Tracing Linux Process Behavior with ptrace: Building and Evaluating a Syscall-Level Forensics Tool

Andrew Gruenewald **Computer Science - Cybersecurity** **October 2026**

This article evaluates a Linux syscall tracer I wrote in C for Technica 2026, where it won the Cipher Tech Best Digital Forensics Hack. The tracer launches one program under ptrace, stops it at every system call entry and exit, and prints each call with selectively decoded arguments and its result. The hackathon implementation is frozen at tag `v1.0-technica`, the question here is narrower than whether the tool works but rather when a process is observed at the syscall boundary, what can an analyst actually reconstruct about its behavior, and where does that evidence stop? Sections 1 and 2 explain why that boundary matters and how the tracer observes it. Section 3 describes the output, Section 4 tests it against workloads with known ground truth, and Section 5 sorts the limits it exposed by their source.

Repository: [andrew04-cybersec/Linux-Syscall-Tracer]
Tag: `v1.0-technica`
Commit: `36fc3c0ab3e413a6ea8f8750759d2fa9479c1ad3`
Evaluation evidence (published separately, not part of `v1.0-technica`): [evaluation/2026-10-25-ptrace-article/], tag `eval-2026-10-25`, commit `695c22e9a8e3c9c7ea42a5751c127b9f56d48b63`

Contents

1. Why Syscall Visibility Matters in Digital Forensics
2. ptrace Architecture
3. What the Tool Does
4. Behavioral Case Study
5. Limitations and the Path to eBPF

References

Appendix: Reproducing the Case Study

---

1. Why Syscall Visibility Matters in Digital Forensics

A system call (syscall) is the interface through which a running program asks the Linux kernel to do work on its behalf, and the Linux man-pages describe it as the fundamental interface between an application and the kernel [2, `syscalls(2)`]. Most programs never issue one directly; they call a C library wrapper that places the syscall number and arguments in registers, traps into kernel mode where the kernel performs the operation, and reports any failure through `errno`, so from the programmer's side a system call looks like an ordinary function call [2, `intro(2)`].

That boundary itself is what matters for forensics because a program just sitting there on disk doesn’t do anything until it runs, and as Forrest et al. put it when they made system calls the basis of their intrusion-detection work, "system damage is caused by running programs that execute system calls" [3, §3]. Files, processes, memory mappings and network connections are resources the kernel manages, so a process has to ask for them; even a statically linked binary, which carries its own copy of libc and does not rely on any external wrapper to make system calls [4, §II.B], still has to enter the kernel for those operations, apart from exceptions covered in Section 5. Watching that entry point shows what a process asked the kernel to do, whether or not the investigator has its source code.

The calls themselves map directly onto questions an investigator asks. `execve` replaces the calling process's program with the one at a given path and does not return on success, which records what was launched and with which arguments [2, `execve(2)`]. `openat` resolves a path and returns a new file descriptor [2, `open(2)`]. `read` and `write` move up to a requested number of bytes between a file descriptor and a buffer and return how many bytes were actually transferred [2, `read(2)`, `write(2)`]. `socket` creates a communication endpoint and returns a descriptor for it, and `connect` connects that socket to the address the caller specifies [2, `socket(2)`, `connect(2)`].

Individually these are ordinary operations, and the evidence comes from how they combine. Consider an illustrative sequence, not a captured trace, in which a binary launched from `/tmp` opens a private SSH key, reads 1,675 bytes from it, creates a TCP socket, connects it, and then writes 1,675 bytes to that socket. What the sequence establishes is observed behavior: which file was opened, how much was read, that a network connection was made, and how much was written to it. What it does not establish is intent, because matching byte counts do not prove that the same bytes left the machine, and a legitimate synchronization tool could produce the same pattern. The sequence supports a hypothesis of credential exfiltration that other evidence would have to confirm. It also shows why arguments matter as much as call names, since Wagner and Soto point out that a monitor which ignores parameters cannot tell a harmless open of a shared library from an open of `/etc/shadow` for writing [5, §3], which is why my tracer decodes path strings rather than printing only call names.

This kind of visibility has a long research history. Forrest et al. showed that short sequences of system calls form a stable signature of a program's normal behavior, and they used departures from that signature to detect attacks against sendmail and lpr [3, §1, §4]. More recently, Cozzi et al. analyzed 10,548 Linux malware samples by recording every system call along with its arguments and return value during five-minute runs, and used those traces to study persistence, process renaming and privilege-dependent behavior; of the 2,637 samples they re-ran as root, 89% behaved differently [4, §III.D, §V.D]. Static inspection is not always an option, since some samples had ELF headers damaged enough that readelf failed on 166 of 211 and GDB on 157 [4, §V.A].

Syscall visibility therefore provides useful behavioral evidence, but it has a clear boundary. A trace represents only what a process was observed asking the kernel to do during one particular execution, so it cannot by itself establish why the process acted, and it does not provide complete visibility into everything the process did. My tracer narrows that view further, because it follows a single process and captures a single run. The specific blind spots at the syscall boundary, and how they shape the rest of this project, are the subject of Section 5.

2. ptrace Architecture

Observing that boundary from user space requires a way to pause another process at each kernel entry and exit and read its registers and memory. On Linux, `ptrace(2)` provides that.

The tracer is a C program for x86-64 Linux built on the relationship `ptrace(2)` defines between a tracer, which observes and controls another process, and a tracee, whose registers and memory the tracer may examine while it is stopped [2]. `main()` calls `fork()`, and the child immediately calls `ptrace(PTRACE_TRACEME, 0, NULL, NULL)`, which makes it a tracee of its parent [1]. `PTRACE_TRACEME` does not stop the caller, so the child next calls `raise(SIGSTOP)`; because a tracee stops whenever a signal is delivered to it, the child enters a signal-delivery-stop, the parent collects that stop with `waitpid()`, and from then on the child runs only when the parent restarts it [2, `ptrace(2)`].

The pause lets configuration happen before the target program runs. The child calls `execvp(argv[1], &argv[1])` only after it is resumed, so the parent, having confirmed with `WIFSTOPPED` that the child is stopped, uses that window to call `PTRACE_SETOPTIONS` with `PTRACE_O_TRACESYSGOOD` and `PTRACE_O_TRACEEXEC`. Because the options are in place before `execvp()` runs, the target's own `execve`, including the failed attempts `execvp()` makes while searching PATH, appears in the trace [2, `exec(3)`]. `PTRACE_O_TRACEEXEC` also changes how a successful exec is reported: without it the kernel sends the tracee a plain `SIGTRAP` after `execve` succeeds, whereas with it the kernel reports a `PTRACE_EVENT_EXEC` stop and no longer generates that legacy `SIGTRAP` [2, `ptrace(2)`].

`trace_loop()` then repeats one cycle: restart the tracee with `PTRACE_SYSCALL`, block in `waitpid()` until it stops, classify the stop, inspect the tracee if warranted, and restart. `PTRACE_SYSCALL` resumes the tracee like `PTRACE_CONT` but arranges for it to stop at the next entry to or exit from a system call, so each completed syscall normally yields two stops [2, `ptrace(2)`]. Without `PTRACE_O_TRACESYSGOOD` a syscall stop reports the same `SIGTRAP` as a genuine signal; with it the kernel sets bit 7 of the signal number, so `WSTOPSIG(status)` returns `SIGTRAP|0x80`, and the loop treats only that value as a syscall stop. An exec event still reports plain `SIGTRAP` but carries `PTRACE_EVENT_EXEC` in the upper bits of the status word, so the loop checks `status >> 16` and treats any nonzero value as an event stop that prints nothing.

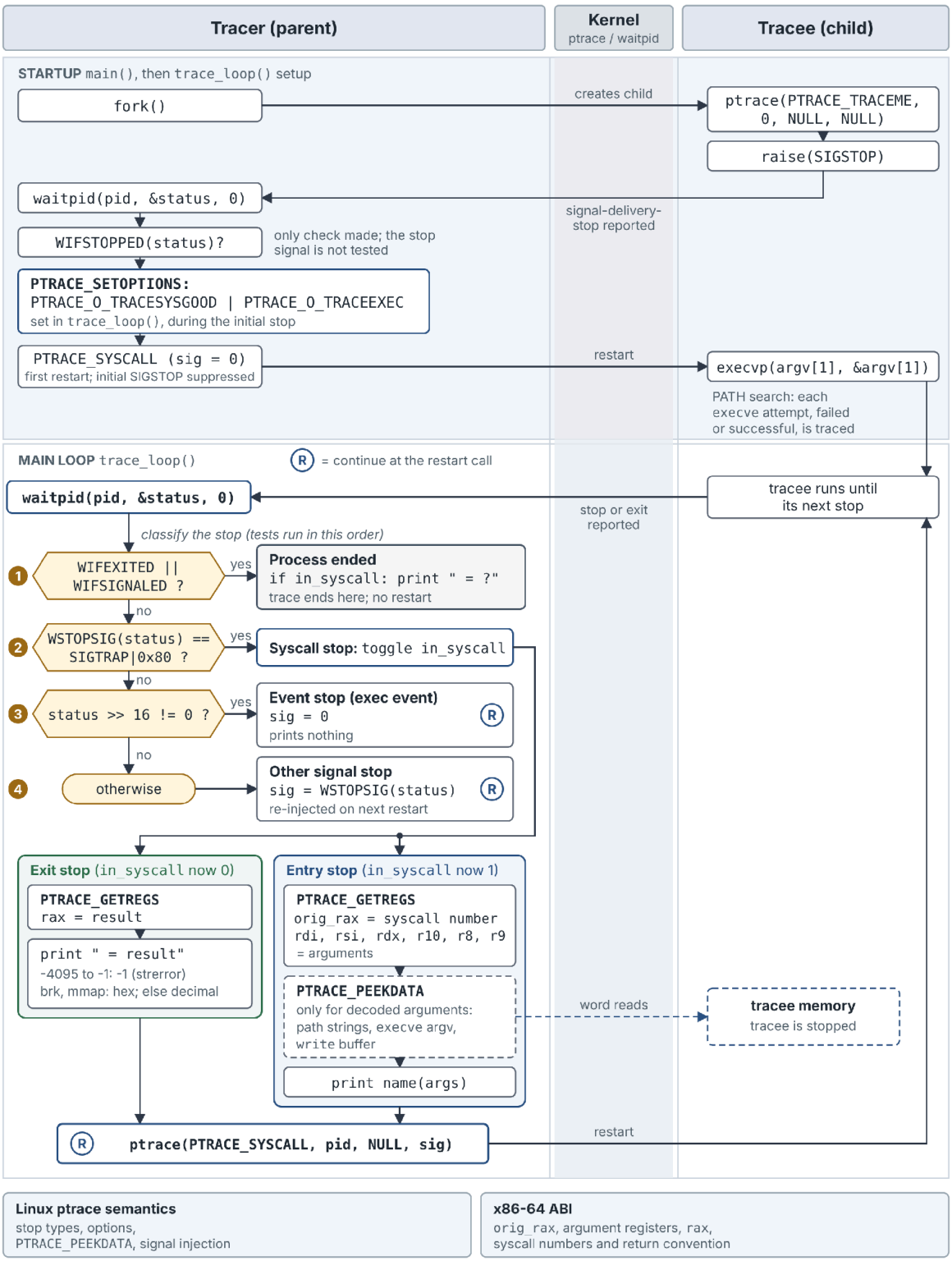


Figure 1. Control flow of the frozen tracer, the child stops itself before `execvp()` so that options are in place before the target's first `execve`; every later stop is classified in the order shown.

Syscall-enter-stops and syscall-exit-stops (entry stops and exit stops from here on) look identical to the tracer, and `ptrace(2)` leaves it to the tracer to keep track of the sequence [2]. The implementation keeps a single `in_syscall` flag and flips it on every `SIGTRAP|0x80` stop, so stops are interpreted as alternating entry and exit. At each one, `PTRACE_GETREGS` copies the tracee's registers into a `struct user_regs_struct` from `<sys/user.h>` [2, `ptrace(2)`]. At entry the syscall number comes from `orig_rax` rather than `rax`: the x86-64 convention passes the number in `rax` [6, A.2.1], but on x86 `rax` already holds `-ENOSYS` at the entry stop and is later reused for the result [2, `ptrace(2)`], so the tracer reads the preserved copy in `orig_rax`, which the kernel uses on entry to hold the syscall number [7, `arch/x86/include/asm/ptrace.h`]. The number is looked up in a 68-entry name table of x86-64 syscalls in `syscalls.c`, and a number not in the table prints as `syscall_N` with its arguments elided. The arguments come from `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9` in that order [2, `syscall(2)`]; the fourth is `r10` rather than the `rcx` of the ordinary function-call convention because the kernel interface substitutes `r10` for `rcx` [6, A.2.1; 7, `arch/x86/entry/entry_64.S`].

Many of those values are pointers into the tracee's address space, which after `fork()` is separate from the tracer's [2, `fork(2)`], so dereferencing one in the tracer would read the tracer's own memory or fault. The tracer instead asks the kernel to read the tracee's memory with `PTRACE_PEEKDATA`, which returns one word, eight bytes on x86-64, per call. Because a successful read can legitimately return -1, `read_string()` and `read_bytes()` clear `errno` before each call and treat -1 as an error only if `errno` was then set, which is the convention `ptrace(2)` prescribes [2]. `read_string()` reassembles a C string word by word, stopping at a NUL byte or the 64-byte buffer limit (63 characters plus the terminator).

Decoding is selective: pathname arguments for a fixed list of 14 syscalls such as `open`, `openat`, `stat`, `execve` and `readlink`; the argv array of `execve`, up to 16 entries; and the buffer passed to `write`, up to 63 bytes using the count argument as its length. Every other argument, including flags, structures, envp and read buffers, is printed as raw hexadecimal. All of this happens at the entry stop, before the kernel acts on the referenced memory, and for argv it is required, because once `execve` succeeds the old address space and its strings no longer exist [2, `execve(2)`].

Restarting with `PTRACE_SYSCALL` produces the matching exit stop, where the tracer reads `rax`, which only now holds the result. Under the x86-64 kernel convention a value between -4095 and -1 is a negated error number [6, A.2.1], which the tracer renders as -1 with the `strerror()` text, as Section 3 describes.

Not every stop is a syscall stop. When `WSTOPSIG(status)` is not `SIGTRAP|0x80` and `status >> 16` is zero, the tracer treats the stop as a signal-delivery-stop, in which the signal reaches the tracee only if the tracer passes it in the data argument of the next restart [2, `ptrace(2)`]. The tracer stores `WSTOPSIG(status)` and re-injects it on the next `PTRACE_SYSCALL`, because passing 0 would silently suppress the signal and change how the traced program behaves; syscall stops, event stops and the initial `SIGSTOP` stop are resumed with 0. The loop ends when `waitpid()` reports `WIFEXITED` or `WIFSIGNALED` [2, `wait(2)`], and since `exit_group` ends the process without an exit stop, a final call whose entry was printed without an exit is closed with `= ?`.

The architecture rests on two layers: the stop model, the options, `PTRACE_PEEKDATA` and signal injection are Linux ptrace semantics, which `ptrace(2)` describes as highly specific to the operating system and architecture [2], while reading `orig_rax` as the syscall number, `rdi` through `r9` as arguments, `rax` as the result with its -4095 to -1 error range, and the numbers in `syscalls.c` depends on the x86-64 syscall ABI. Because no fork, vfork or clone tracing options are set, the tracer follows exactly one process. Section 3 turns from the mechanism to what the analyst actually receives.

3. What the Tool Does

The analyst runs the tracer by placing it in front of an ordinary command, as in `./tracer cat /etc/hostname`. The words after the tracer's own name are passed to `execvp()` as the target's argv, and `execvp()` searches PATH when the program name contains no slash [2, `exec(3)`]. If no command is given, the tracer prints a usage line and exits with status 1. What comes back is a chronological log of the system calls one process made during one run. The tracer writes it unbuffered to standard error, which keeps it separate from the target's standard output and lets the analyst capture it with `2> trace.log`. The target inherits the same standard error, so anything the target writes there lands in the same file and can interleave with the trace, even mid-line, because each call is printed at entry and its result at exit.

Every recorded call follows one pattern, `name(arguments) = result`, with no process ID, timestamp or indentation [1]. The name comes from a table of 68 x86-64 syscalls in `syscalls.c`, and the table also supplies how many arguments each call takes, so the tracer prints only the arguments that call actually uses. A number that is not in the table still produces a line, but in the form `syscall_N(...)` with its arguments elided, so the analyst always sees that a call happened. That matters because modern programs routinely make calls outside the table; in verification runs `clock_nanosleep` appears as `syscall_230`, and `getsockopt`, `getsockname` and `getpeername` appear as `syscall_55`, `syscall_51` and `syscall_52` [7, `arch/x86/entry/syscalls/syscall_64.tbl`; 8]. For a named call without specialized decoding, every argument is printed as a raw hexadecimal value, so a file descriptor appears as `0x3`, flags appear as bit patterns such as `0x80000`, and a pointer appears as an address with nothing read from behind it.

On top of that baseline the tracer decodes three kinds of argument, each of which can be read as a bounded string and carries information an analyst often needs. The first is pathnames. For 14 syscalls the tracer follows the path pointer and prints the string it finds: argument 0 for `open`, `access`, `execve`, `stat`, `lstat`, `statfs`, `chdir`, `unlink`, `readlink`, `lgetxattr` and `listxattr`, and argument 1 for `openat`, `newfstatat` and `statx`, whose first argument is a directory descriptor. A line such as `openat(0xffffff9c, "/etc/hostname", 0x0, 0x0) = 3` shows the requested path and the returned descriptor, while the directory argument and flags stay hexadecimal. Strings are shown up to 63 characters, with a `...` after the closing quote when they were cut off, quotes, backslashes and newlines escaped, other non-printable bytes written as `\xNN`, and the raw pointer printed instead if the memory cannot be read.

The second is `execve`, where the tracer decodes both the path and the argv array, showing up to 16 entries and then `...` if more remain, while envp stays a raw pointer. Because `execvp()` tries each PATH directory in turn, the trace records every failed attempt as `execve(...) = -1 (No such file or directory)` before the successful call returns 0, showing both what was launched and where the system looked. The third is `write`, where the tracer reads the buffer using the count argument as its length, shows at most 63 bytes with the same escaping, and marks longer writes with `...` while the count and return value still report the full size. No other output call receives this treatment: `writev` and `pwrite64` are named but print their buffers as pointers, and `sendto` is not in the table at all.

Results are printed when the call finishes. Ordinary successes appear in decimal, so descriptor numbers and byte counts read directly, while `brk` and `mmap` results appear in hexadecimal because they are addresses. A kernel error is rewritten the way the C library would report it, as -1 followed by the `strerror()` text, so failed lookups, permission denials and refused connections read as plain English. When the process ends while a call is still outstanding, as with `exit_group` or a process killed during a call, the last line ends in `= ?`. Signals themselves are not printed, and the tracer prints no exit-status summary or signal-termination marker, although a program ending through `exit_group` shows its requested exit code as that call's raw argument, as in `exit_group(0x2)`; the tracer's own exit status is 0 whether the target succeeded or failed.

From one execution, the analyst can read off which program was launched with which arguments and through which PATH lookups, which paths were opened, checked or removed and whether each attempt succeeded, which descriptors were read from or written to and how many bytes moved each time, the leading bytes of anything sent through `write`, and when `socket` and `connect` were called and whether they succeeded. What the trace does not record is equally specific: the contents of read buffers, the destination passed to `connect`, which appears only as a pointer and a length, and anything done by child processes or threads. In a verification run, the parent's `vfork` and `wait4` calls appeared in the trace, but none of the child's own calls did [8]. Linking a descriptor to a file or socket is the analyst's inference from earlier `openat` or `socket` lines, and a `write` that follows a `read` is evidence of a sequence, not proof that the same data moved. Section 4 tests how far that output goes by running workloads whose behavior was fixed in advance.

4. Behavioral Case Study

Environment

Item	Value
Architecture	x86_64
Distribution	Ubuntu 24.04.5 LTS (cloud container)
Kernel	Linux 6.18.44-fc-v153
C library	GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.9) 2.39
Compiler (tracer and helpers)	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
coreutils (`cat`)	GNU coreutils 9.4

Item	Value
Tracer	tag `v1.0-technica`, commit `36fc3c0ab3e413a6ea8f8750759d2fa9479c1ad3`, built with the repository Makefile
`/etc/hostname` content	`vm\n` (3 bytes)

Runs. Primary workloads A, B and C, and the two Workload A stdout checks, were each started with `env -i PATH=/usr/local/bin:/usr/bin:/bin`. Supporting run D (interrupted sleep) was recorded as `./tracer sleep 2 2>D.trace &`, with SIGTSTP sent to the traced sleep after 0.5 s; its recorded command has no `env -i` prefix, so the clean-environment condition is not claimed for run D.

Where these traces come from. All traces in this section were collected in this Ubuntu cloud environment, not on the WSL2 environment where the tracer was originally developed. Loader prefix length, PIDs, addresses and possibly `cat`'s copy strategy may differ elsewhere.

The case study ran three workloads whose behavior was fixed in advance, so each trace could be read against a known ground truth. Workload A was `cat /etc/hostname`, Workload B was a small C program that creates `/tmp/tracer_case_study.txt` and writes the string `FORENSICS_TEST_2026` followed by a newline, and Workload C was a small C program that attempts a TCP connection to port 9 on 127.0.0.1, where nothing was listening. All three ran under the frozen `v1.0-technica` build on x86-64 Ubuntu 24.04.5 with Linux 6.18.44 and glibc 2.39, with the helper programs compiled by gcc 13.3.0 and each primary run started from an empty environment with PATH set to `/usr/local/bin:/usr/bin:/bin`. A supporting run interrupted a traced `sleep 2` with SIGTSTP to check an earlier error 516 observation. Trace output went to a file through stderr, and for each run I separated what the workload was told to do, what the trace records, what an analyst can infer by connecting records, and what the trace leaves unknown [8]. These results describe one environment and should not be read as universal.

4.1 Workload A: File access

Workload A was run with stdout piped to another process, the closest match to the demo setup. The trace has 46 records, and the first two show `execvp()`'s PATH search directly: an `execve` of `/usr/local/bin/cat` that fails with No such file or directory, followed by a successful `execve` of `/usr/bin/cat` with argv decoded as `["cat", "/etc/hostname"]`. The next 31 records are the dynamic loader and C library starting up, after which the behavior of interest appears as a short contiguous run.

```
execve("/usr/local/bin/cat", ["cat", "/etc/hostname"], 0x7ffe1a14a8d8) = -1 (No such file or
   directory)
execve("/usr/bin/cat", ["cat", "/etc/hostname"], 0x7ffe1a14a8d8) = 0
[...]
fstat(0x1, 0x7fffe7d105f0) = 0
openat(0xffffff9c, "/etc/hostname", 0x0, 0x0) = 3
fstat(0x3, 0x7fffe7d105f0) = 0
syscall_221(...) = 0
mmap(0x0, 0x22000, 0x3, 0x22, 0xffffffff, 0x0) = 0x7fec6ebde000
read(0x3, 0x7fec6ebdf000, 0x20000) = 3
write(0x1, "vm\n", 0x3) = 3
read(0x3, 0x7fec6ebdf000, 0x20000) = 0
munmap(0x7fec6ebde000, 0x22000) = 0
close(0x3) = 0
close(0x1) = 0
close(0x2) = 0
exit_group(0x0) = ?
```

Excerpt 1. Workload A, `cat /etc/hostname` with stdout to a pipe (A2.trace lines 1 to 2 and 34 to 46). `[...]` omits 31 loader and libc startup records. The PATH search, open, reads and write are all in order; that descriptor 3 is `/etc/hostname` is not stated anywhere in the trace.

The trace directly records the requested path, the descriptor the kernel returned for it, two reads on descriptor 3 that returned 3 bytes and then 0, a write of `"vm\n"` to descriptor 1, and a close of descriptor 3. The analyst can infer that the reads consumed `/etc/hostname`, that the zero-byte read is end of file, and that the write sent the file's contents to standard output, but each of those steps is a correlation made by the analyst. The tracer never states that descriptor 3 refers to `/etc/hostname`, and the correlation needs care, because the same number 3 had already been used and closed twice during startup for `/etc/ld.so.cache` and `libc.so.6`.

The read buffers themselves are only addresses, so the bytes read are known by count, and the content appears only because the program happened to write it out again. The call numbered 221 in the trace, which is `fadvise64` on x86-64 [7, `arch/x86/entry/syscalls/syscall_64.tbl`], appears as `syscall_221(...)`, so its arguments are not visible.

An unplanned variation, Workload A (stdout to a file), showed how much the trace depends on the workload's surroundings. When the same command ran with stdout redirected to a regular file instead of a pipe, `cat` made no `read` or `write` calls at all. In their place the trace recorded `syscall_326(...) = 3` followed by `syscall_326(...) = 0`, which is `copy_file_range` [7, `arch/x86/entry/syscalls/syscall_64.tbl`], so the copy happened inside the kernel through a call the tracer does not name. That trace still shows the open of `/etc/hostname`, but neither the destination descriptor nor the copied content, only that 3 bytes moved.

```
openat(0xffffff9c, "/etc/hostname", 0x0, 0x0) = 3
fstat(0x3, 0x7ffdde232c30) = 0
syscall_221(...) = 0
uname(0x7ffdde232cc0) = 0
syscall_326(...) = 3
syscall_326(...) = 0
close(0x3) = 0
```

Excerpt 2. Workload A (stdout to a file), A.trace lines 35 to 41. Syscall 326 is `copy_file_range`, which is not in the tracer's 68-entry name table. This behavior was observed with GNU coreutils 9.4 writing to a regular file and is not a general property of `cat`.

4.2 Workload B: File creation and write

Workload B was compiled as a small dynamically linked program, which kept the trace to 33 records, 28 of them loader startup between the `execve` and the program's own first call.

```
openat(0xffffff9c, "/tmp/tracer_case_study.txt", 0x241, 0x1a4) = 3
write(0x3, "FORENSICS_TEST_2026\n", 0x14) = 20
close(0x3) = 0
exit_group(0x0) = ?
```

Excerpt 3. Workload B, B.trace lines 30 to 33: the program's own calls after 28 loader and libc startup records. Path, descriptor, payload and byte count are printed as text; the file's final on-disk contents are known only from ground truth.

The trace records the full sequence the program was told to perform. The pathname is decoded, the call returns descriptor 3, the write on descriptor 3 shows the payload `FORENSICS_TEST_2026\n` with a requested count of `0x14` and a result of 20 bytes, and descriptor 3 is then closed. Write decoding is what makes this readable evidence, because the content appears in the trace without examining the file afterwards. Two parts still rely on interpretation. The flags `0x241` and mode `0x1a4` are printed raw, and an analyst has to decode them as `O_WRONLY | O_CREAT | O_TRUNC` with permissions `0644`, and the trace never directly states that the write landed in `/tmp/tracer_case_study.txt`. That conclusion comes from correlating the returned descriptor with the write that followed it, which is straightforward here only because nothing reopened descriptor 3 in between. The program called `open()`, but the trace shows `openat`, which is how this C library implements `open()`.

4.3 Workload C: Network connection attempt

Workload C had the same 28-record startup prefix, followed by the network behavior and an exit.

```
socket(0x2, 0x1, 0x0) = 3
connect(0x3, 0x7ffdfa8a5920, 0x10) = -1 (Connection refused)
close(0x3) = 0
exit_group(0x2) = ?
```

Excerpt 4. Workload C, C.trace lines 30 to 33. The destination, 127.0.0.1 port 9, is known from the experiment's ground truth; the trace shows only a pointer and a 16-byte length.

The trace records that the process created a socket, that the call returned descriptor 3, that `connect` on descriptor 3 failed with Connection refused, and that the descriptor was then closed. The raw arguments `0x2` and `0x1` correspond to `AF_INET` and `SOCK_STREAM`, and the length `0x10` matches the 16-byte size of an IPv4 address structure, so an analyst can infer an attempted IPv4 TCP connection that the far side refused. The destination itself is not in the trace. The second argument of `connect` is a pointer into the tracee's memory, and the tracer prints that address without reading the structure behind it, so neither 127.0.0.1 nor port 9 can be recovered from the output, even though both are known from the experiment's ground truth. This workload made no other socket calls, so it did not exercise the unnamed socket-related calls seen in Section 3's verification runs. The final

`exit_group(0x2)` records that the program requested exit status 2; the tracer does not independently report the final process status.

4.4 Cross-workload analysis

Table 1. Descriptive counts per primary workload (not detection metrics).

Measure	A	B	C
Syscall records	46	33	33
`syscall_N` records	1	0	0
Failed results	2	1	2
Startup records before workload	31	28	28
Decoded paths / write buffers	6 / 1	5 / 1	4 / 0
Central behavior visible	Yes	Yes	Yes, without destination

In every case the central behavior was directly visible and appeared in order, and in every case it sat at the end of a much larger block of loader and libc startup records, between 28 and 31 records, that the analyst has to recognize and set aside. The easiest information to reconstruct was what the selective decoding prints as text: the executable and its argv, the PATH search, the paths opened, and the content of write calls. Everything that connects those records into a behavior, in particular which file or socket a descriptor refers to, was manual correlation, made harder by the reuse of descriptor 3 across startup and the workload. Raw hexadecimal flags, socket families and lengths could be decoded with header knowledge, but the `connect` destination could not be decoded from the output at all. Unknown syscalls were rare in the primary runs, but where they appeared, as with `fadvise64` and `copy_file_range`, they hid exactly the arguments needed to follow the data. Run D (interrupted sleep) reproduced the Unknown error 516 observation from earlier testing, and it is left for Section 5.

The case study shows that the frozen tracer exposes ordered behavioral evidence from a controlled execution, enough to reconstruct each workload's intended behavior once descriptors were correlated by hand. It also shows where that evidence stops, and Section 5 sorts those stopping points by where they come from.

5. Limitations and the Path to eBPF

The tracer did what it was built to do: it recorded one process's syscalls in order, with enough decoding to reconstruct each Section 4 workload's intended behavior. Evaluating that record against known ground truth also exposed the boundary between a useful forensic tracer and the requirements of broader endpoint telemetry. That boundary has three distinct sources: this frozen implementation, ptrace's collection model, and the syscall boundary itself. Some observations, such as `cat` switching to `copy_file_range`, also depend on the tested environment or workload; they qualify the evidence rather than form a fourth source, and they are labeled where they appear.

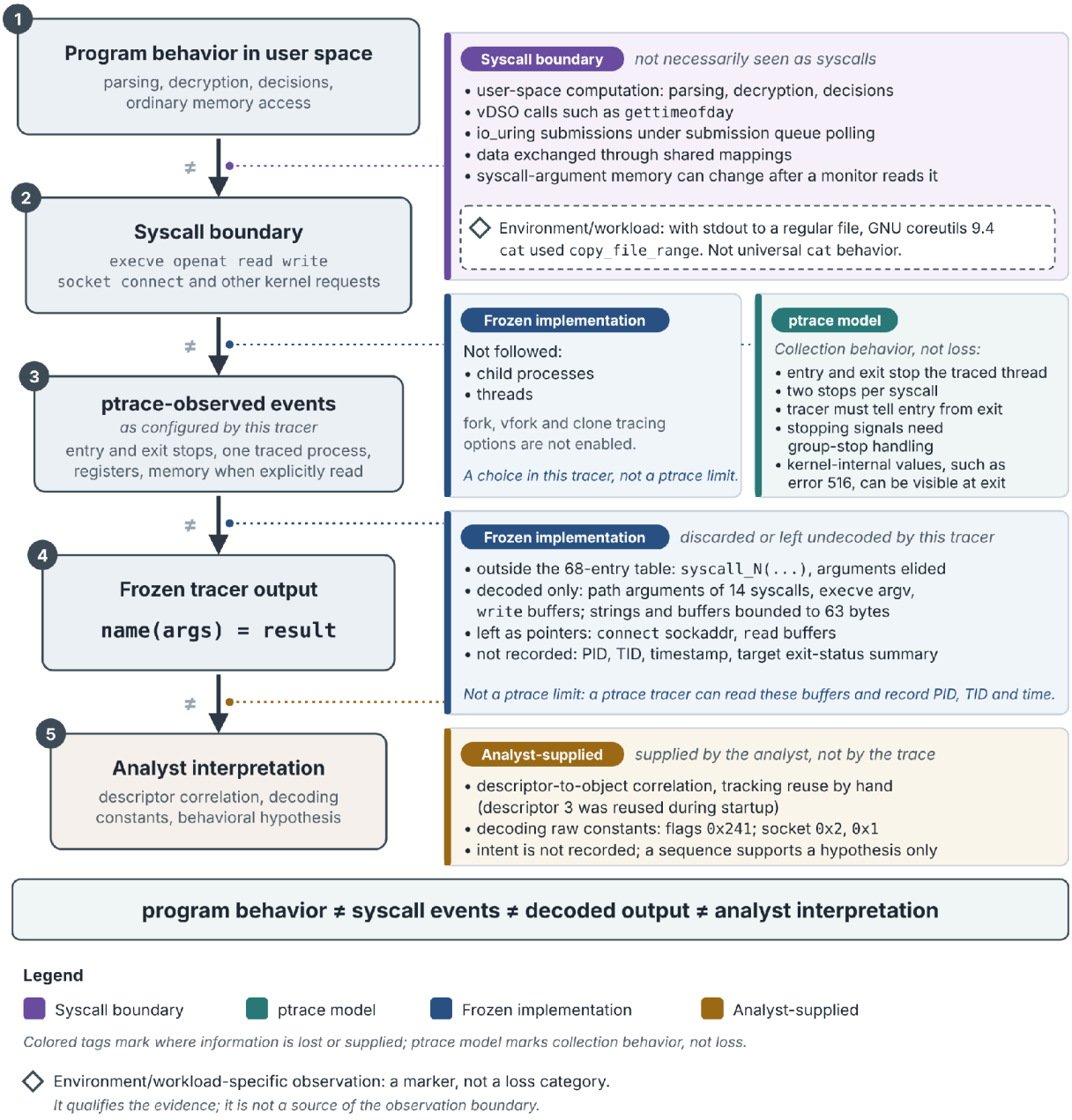


Figure 2. What a syscall trace can and cannot carry (each layer carries less than the one above it), tags mark whether a loss comes from the syscall boundary, ptrace's collection model, this frozen implementation, or the analyst, and a separate marker flags results that depend on the tested environment or workload.

5.1 Limits of the frozen implementation

Most of what the case study could not show traces to decisions in `tracer.c` and `syscalls.c`, not to ptrace [1]. The name table holds 68 entries, and every other syscall prints as `syscall_N` with its arguments elided. The `copy_file_range` result in Section 4 (Excerpt 2) is the clearest case. That is an observation about this coreutils build, not a property of `cat`, but it shows how quickly a fixed table falls behind real syscall selection.

Pointer decoding is equally selective: paths for 14 syscalls, `execve` argv and write buffers, with every other pointer printed as an address. The case study therefore exposed a distinction between observing a socket operation and recovering its destination: `connect(0x3, 0x7ffdfa8a5920, 0x10)` proves a refused connection attempt but never reveals 127.0.0.1 or port 9. Likewise, `read` showed a descriptor, buffer address, size and byte count, and the contents of `/etc/hostname` appeared only because `cat` later passed them to `write`. `PTRACE_PEEKDATA` could have read either buffer at the appropriate stop; the tracer never asks.

The tracer also keeps no state, so every descriptor in Section 4, including the reused descriptor 3, had to be resolved by hand. Records carry no PID, TID, timestamp or schema. Because `trace_loop()` sets only `PTRACE_O_TRACESYSGOOD` and `PTRACE_O_TRACEEXEC`, children and threads are not followed; the vfork check in Section 3 shows this for a child process. `exit_group(0x2)` shows the requested status, but the tracer never reports what `waitpid()` returned. None of these is a ptrace limitation: `ptrace(2)` documents `PTRACE_O_TRACEFORK`, `PTRACE_O_TRACEVFORK` and `PTRACE_O_TRACECLONE` for automatically attaching new children and threads [2], and `strace(1)` shows that a ptrace tool can decode structures, follow forks and timestamp output [9].

5.2 Limits of ptrace-style endpoint collection

Other limits come from ptrace's control model and would remain in a far more complete tracer. `ptrace(2)` describes a tracee that stops, is reported through `waitpid()`, is inspected while stopped, and is then explicitly restarted; under `PTRACE_SYSCALL` this happens at entry and again at exit, and the manual notes that syscall-stops occur twice per system call [2]. Each observed syscall therefore costs two stops, two wait notifications, register reads, any `PTRACE_PEEKDATA` words needed for decoding and two restarts, while the traced thread waits on the tracer. That overhead grows with the amount of traced activity, though this frozen tracer was not benchmarked. `strace(1)` itself lists slow traced processes under BUGS and offers seccomp-bpf filtering, which reduces stops without changing the model [9].

The model also makes the tracer responsible for reconstructing kernel state. The `in_syscall` toggle in Section 2 exists because entry and exit stops look identical, and `ptrace(2)` notes that seccomp stops can produce an exit stop without a matching entry stop [2]. Each tracee is one thread, so following a workload means auto-attaching every new task and tracking per-thread state, including across a multithreaded `execve` that resets thread IDs [2, `ptrace(2)`].

Signals add another layer. Run D (interrupted sleep) produced these two records:

```
syscall_230(...) = -1 (Unknown error 516)
syscall_219(...) = 0
```

Excerpt 5. Run D (interrupted sleep), D.trace lines 34 to 35: SIGTSTP sent to a traced `sleep 2`. The signal itself is not printed; that the tracee kept running comes from `/proc`, which showed it in state S rather than T. Single supporting run.

On x86-64 these are `clock_nanosleep` and `restart_syscall` [7, `arch/x86/entry/syscalls/syscall_64.tbl`], and 516 is `ERESTART_RESTARTBLOCK`, a kernel-internal code that kernel source says user programs should never see but ptrace can observe at syscall exit [7, `include/linux/errno.h`]; the `restart_syscall` that follows is consistent with the

restart mechanism `ptrace(2)` and `restart_syscall(2)` describe [2]. SIGTSTP also failed to leave the process stopped, consistent with a pitfall `ptrace(2)` documents: a tracer restarts a group-stopped tracee intending to inject the stop signal, and the tracee continues to run. Transparent handling of stopping signals relies on `PTRACE_LISTEN`, which works only on tracees attached with `PTRACE_SEIZE` [2, `ptrace(2)`]. None of this is a vulnerability, the 516 result is kernel behavior that ptrace exposes; the SIGTSTP result is that documented pitfall occurring in this tracer, which is attached through `PTRACE_TRACEME` and does not distinguish a group-stop from a signal-delivery-stop. Both show how much kernel semantics a transparent ptrace tool must model.

Attachment is also gated: `PTRACE_ATTACH` and `PTRACE_SEIZE` require a ptrace access mode check based on credentials, dumpability, `CAP_SYS_PTRACE` and LSMs [2, `ptrace(2)`], and Yama, where it is enabled, can further restrict attachment according to its `ptrace_scope` policy [10]. ptrace remains appropriate for controlled tracing of a chosen program, but a model that stops individual threads until a user-space tracer restarts them is a poor fit for continuous collection across a host.

5.3 The syscall boundary is not the whole process

Even perfect syscall interception sees only what crosses into the kernel. Parsing, decryption and decision logic run in user space and produce no syscall record. `vdso(7)` states that vDSO-exported functions such as `gettimeofday` do not appear in strace output [2]. With io_uring, setup and `io_uring_enter` remain syscalls, but each submission queue entry is in essence the equivalent of a system call, and with submission queue polling the application can avoid calling `io_uring_enter` [11]. Processes sharing a mapping exchange data through ordinary memory access once the mapping syscalls complete [2, `mmap(2)`, `shm_overview(7)`]. Garfinkel also described how shared memory referenced by syscall arguments can change between a monitor's read and the kernel's use [12, §4.3.3]. A syscall trace is therefore evidence at a specific observation boundary: strong for kernel-mediated file, process and network operations, and silent about the rest.

5.4 Why the next project moves to eBPF

These findings define what the next project, a Linux endpoint telemetry and behavioral detection engine, must investigate, and eBPF is a reasonable mechanism to explore. Programs loaded through `bpf(2)` pass an in-kernel verifier and attach to events such as tracepoints [2, `bpf(2)`], and a tracepoint's attached function runs in the execution context of the caller instead of stopping the task for a separate tracer [13]. On kernels with syscall tracepoints, `raw_syscalls:sys_enter` records the syscall number and six raw argument values and `sys_exit` the return value, for any calling task; `sched_process_fork` and `sched_process_exec` mark lifecycle transitions [7, `include/trace/events/syscalls.h`, `include/trace/events/sched.h`]; `bpf_get_current_pid_tgid` supplies PID and TGID (the thread-group ID) [2, `bpf-helpers(7)`]; and maps and the BPF ring buffer deliver structured records to user space [14].

This changes the engineering problem rather than eliminating it. The verifier bounds analysis at one million explored instructions [15], and in Linux v6.18 source `MAX_BPF_STACK` limits a BPF program to 512 bytes of stack [7, `include/linux/filter.h`]; loading and attaching tracing programs is privileged, with requirements that vary by kernel version, configuration and attachment mechanism [7, `kernel/bpf/syscall.c`, `kernel/bpf/token.c`]; and the kernel's BPF Design Q&A states that

neither tracepoints nor kprobe attachment sites are a stable ABI [15]. A full ring buffer fails new reservations rather than blocking, so event loss must be measured [14]. Many raw syscall arguments are still user-space pointers to decode, probe selection still defines the observation boundary, and every probe adds cost.

Each tracer lesson therefore becomes a requirement, not a solved problem.

Table 2. Planned project: requirements for a future Linux endpoint telemetry and behavioral detection engine using eBPF. None of these components exists yet.

Observed in this evaluation	Requirement for the planned project
Single-process view	Collect events across processes and maintain fork, exec and exit relationships in a process graph.
Manual descriptor correlation	Build explicit per-process state mapping descriptors to files and sockets, which no probe supplies.
Raw, partially decoded arguments	Normalize events into a versioned schema with PID, TID and timestamps.
Hidden sockaddr and `copy_file_range` arguments	Select probes and decoders deliberately and test them against workload variation.
Behavior requiring human interpretation	Implement explicit, stateful detections mapped to ATT&CK rather than treating telemetry as detection.
Unmeasured cost and accuracy	Evaluate latency, event loss, false positives and coverage against controlled attack simulations and benign workloads.

The ptrace tracer showed how Linux behavior appears at the syscall boundary. The Section 4 evaluation measured what that representation reveals and misses, and separated gaps I created from gaps imposed by ptrace and the boundary itself. The next project will treat those results as design requirements: a different collection mechanism, with state, normalization, detection logic and evaluation still the engineer's responsibility.

References

[1] Andrew Gruenewald. *Linux Syscall Tracer*, tag `v1.0-technica`, commit `36fc3c0ab3e413a6ea8f8750759d2fa9479c1ad3`. https://github.com/andrew04-cybersec/Linux-Syscall-Tracer. Pinned source: https://github.com/andrew04-cybersec/Linux-Syscall-Tracer/tree/36fc3c0ab3e413a6ea8f8750759d2fa9479c1ad3. Files: `tracer.c`, `syscalls.c`, `syscalls.h`, `Makefile`. Key locations at that commit (append `#L<line>` to the file permalink):

- `tracer.c`: `fork()` 244; `PTRACE_TRACEME` 252; `raise(SIGSTOP)` 258; `execvp()` 260; initial `waitpid()` 267 and `WIFSTOPPED` check 272; `PTRACE_SETOPTIONS` 166; initial `sig = 0` 173; `PTRACE_SYSCALL` 176 and `waitpid()` 180; exit test 185; `SIGTRAP|0x80` test 188; `in_syscall` toggle 191; `PTRACE_GETREGS` 192 to 193; `orig_rax` 199; result in `rax` 210; result rendering 212 to 218; event stop 222 to 223; signal re-injection 226; `= ?` 231 to 232; `read_string()` 19 to 41; `read_bytes()` 44 to 56; `print_string_array()` 85 to 113; `print_written()` 116 to 126.
- `syscalls.c`: name table entries 13 to 81; `syscall_path_arg()` 106 to 129.

[2] Linux man-pages project, release 6.7. https://man7.org/linux/man-pages/. Pages cited: `syscalls(2)`, `intro(2)`, `syscall(2)` ("Architecture calling conventions"), `execve(2)`, `open(2)`, `read(2)`, `write(2)`, `socket(2)`, `connect(2)`, `exec(3)`, `fork(2)`, `wait(2)`, `ptrace(2)` (DESCRIPTION, `PTRACE_SETOPTIONS`, `PTRACE_PEEKDATA`, `PTRACE_GETREGS`, `PTRACE_LISTEN`, "Signal-delivery-stop", "Signal injection and suppression", "Group-stop", "PTRACE_EVENT stops", "Syscall-stops", "Attaching and detaching", "execve(2) under ptrace", RETURN VALUE, NOTES including ptrace access mode checking), `restart_syscall(2)`, `vdso(7)` (NOTES, "strace(1), seccomp(2), and the vDSO"), `mmap(2)`, `shm_overview(7)`, `bpf(2)`, `bpf-helpers(7)` (`bpf_get_current_pid_tgid`).

[3] Stephanie Forrest, Steven A. Hofmeyr, Anil Somayaji, and Thomas A. Longstaff. "A Sense of Self for Unix Processes." In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, Oakland, CA, May 6-8, 1996, pp. 120-128. IEEE Computer Society. DOI: 10.1109/SECPRI.1996.502675. https://doi.org/10.1109/SECPRI.1996.502675 (open copy: https://www.ieee-security.org/TC/SP2020/tot-papers/forrest-1996.pdf)

[4] Emanuele Cozzi, Mariano Graziano, Yanick Fratantonio, and Davide Balzarotti. "Understanding Linux Malware." In *2018 IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, May 21-23, 2018. IEEE. https://www.s3.eurecom.fr/docs/oakland18_cozzi.pdf

[5] David Wagner and Paolo Soto. "Mimicry Attacks on Host-Based Intrusion Detection Systems." In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS '02)*, Washington, DC, November 18-22, 2002, pp. 255-264. ACM. DOI: 10.1145/586110.586145. https://doi.org/10.1145/586110.586145

[6] H. J. Lu, Michael Matz, Milind Girkar, Jan Hubička, Andreas Jaeger, and Mark Mitchell (eds.). *System V Application Binary Interface, AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models)*, Version 1.0. Appendix A.2.1, "AMD64 Linux Kernel Conventions: Calling Conventions" (marked informative). https://gitlab.com/x86-psABIs/x86-64-ABI

[7] Linux kernel source, tag `v6.18` (commit `7d0a66e4bb9081d75c82ec4957c50034cb0ea449`). https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/?h=v6.18. Files cited: `arch/x86/include/asm/ptrace.h` (`orig_ax` comment in x86-64 `struct pt_regs`); `arch/x86/entry/entry_64.S` ("Registers on entry"); `arch/x86/entry/syscalls/syscall_64.tbl` (51 `getsockname`, 52 `getpeername`, 55 `getsockopt`, 219 `restart_syscall`, 221 `fadvise64`, 230 `clock_nanosleep`, 326 `copy_file_range`); `include/linux/errno.h` (ERESTART comment; `ERESTART_RESTARTBLOCK` 516); `include/trace/events/syscalls.h`; `include/trace/events/sched.h`; `include/linux/filter.h` (`MAX_BPF_STACK` 512); `kernel/bpf/syscall.c`; `kernel/bpf/token.c`.

[8] Evaluation artifacts for this article, recorded October 25, 2026 against the frozen `v1.0-technica` build in the environment described in Section 4: saved traces `A2.trace`, `A3.trace`, `A.trace`, `B.trace`, `C.trace` and `D.trace` with SHA-256 checksums, helper sources `write_case.c` and `connect_case.c`, and the environment record. Published separately from the frozen implementation in `evaluation/2026-10-25-ptrace-article/` in the Linux Syscall Tracer repository, tag `eval-2026-10-25`, commit `695c22e9a8e3c9c7ea42a5751c127b9f56d48b63`. https://github.com/andrew04-cybersec/Linux-Syscall-Tracer/tree/695c22e9a8e3c9c7ea42a5751c127b9f56d48b63/evaluation/2026-10-25-ptrace-article. The Section 3 verification runs were not saved as trace files; their results are reported in the text only.

[9] `strace(1)`, strace project, version 6.8. https://man7.org/linux/man-pages/man1/strace.1.html. Sections: OPTIONS (`-f`, `-t`, `--seccomp-bpf`), BUGS.

[10] Linux kernel documentation, `Documentation/admin-guide/LSM/Yama.rst`, "Yama," section `ptrace_scope`. https://docs.kernel.org/admin-guide/LSM/Yama.html

[11] `io_uring(7)` and `io_uring_sqpoll(7)`, liburing, version 2.5. https://man.archlinux.org/man/io_uring.7.en and https://man7.org/linux/man-pages/man7/io_uring_sqpoll.7.html. Section: Submission queue polling.

[12] Tal Garfinkel. "Traps and Pitfalls: Practical Problems in System Call Interposition Based Security Tools." In *Network and Distributed System Security Symposium (NDSS 2003)*, 2003. https://www.ndss-symposium.org/wp-content/uploads/2017/09/Traps-and-Pitfalls-Practical-Problems-in-System-Call-Interposition-Based-Security-Tools-Tal-Garfinkel.pdf

[13] Mathieu Desnoyers. "Using the Linux Kernel Tracepoints." Linux kernel documentation, `Documentation/trace/tracepoints.rst`. https://docs.kernel.org/trace/tracepoints.html

[14] "BPF ring buffer." Linux kernel documentation, `Documentation/bpf/ringbuf.rst`, section "Semantics and APIs." https://docs.kernel.org/bpf/ringbuf.html

[15] "BPF Design Q&A." Linux kernel documentation, `Documentation/bpf/bpf_design_QA.rst`: verifier limits; stack space; tracepoints and kprobe attachment sites as stable ABI. https://docs.kernel.org/bpf/bpf_design_QA.html

---

Appendix: Reproducing the Case Study

The tracer itself is not modified for any of these steps and the helper programs and commands below are evaluation material, separate from the frozen source.

A.1 Test environment

The case study ran in the environment listed in the Section 4 Environment box: x86_64, Ubuntu 24.04.5 LTS (cloud container), Linux 6.18.44-fc-v153, GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.9) 2.39, gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU coreutils 9.4, and `/etc/hostname` containing `vm\n` (3 bytes). This is not the WSL2 environment the tracer was originally developed on; rerunning there may change loader prefix length, PIDs, addresses and possibly `cat`'s copy strategy.

A.2 Build

```
git -C Linux-Syscall-Tracer archive v1.0-technica | tar x   # into an empty directory
make
gcc -Wall -Wextra -O0 -o write_case write_case.c
gcc -Wall -Wextra -O0 -o connect_case connect_case.c
```

`make` uses the repository Makefile (`gcc -Wall -Wextra -g`). The helpers are dynamically linked and were written in C to keep interpreter startup out of the traces.

A.3 Runs

Run	Command or procedure as recorded	stdout	Trace file
A (primary)	`env -i PATH=/usr/local/bin:/usr/bin:/bin ./tracer cat /etc/hostname 2>A2.trace | od -c`	pipe to `od` (showed `vm\n`)	`A2.trace`
A, `/dev/null` check	Same as A, with stdout redirected to `/dev/null`	`/dev/null`	`A3.trace` (identical to A2.trace apart from a PID)

Run	Command or procedure as recorded	stdout	Trace file
A (stdout to a file)	Same as A, with stdout redirected to the regular file `A.stdout`	`A.stdout` (contained `vm\n`)	`A.trace`
B	`env -i PATH=/usr/local/bin:/usr/bin:/bin ./tracer ./write_case >B.stdout 2>B.trace`	`B.stdout` (empty)	`B.trace`
C	`env -i PATH=/usr/local/bin:/usr/bin:/bin ./tracer ./connect_case >C.stdout 2>C.trace`	`C.stdout` (empty)	`C.trace`
D (supporting)	`./tracer sleep 2 2>D.trace &`, then SIGTSTP sent with `kill -TSTP` to the traced `sleep` after 0.5 s; the exact invocation, including how the `sleep` PID was obtained, was not preserved	terminal	`D.trace`

Run D's recorded command has no `env -i` prefix, so the clean-environment condition is not claimed for it. After SIGTSTP, the traced `sleep`'s process state was read from `/proc` and showed S (sleeping) rather than T (stopped); the exact inspection command was not preserved.

Recorded outcomes: the tracer exited 0 in every run. Run directly, `write_case` exits 0 and `connect_case` exits 2 (connect failed). After run B, `/tmp/tracer_case_study.txt` contained exactly `FORENSICS_TEST_2026` and a newline (20 bytes). Nothing was listening on 127.0.0.1:9 before run C.

A.4 Helper source

The listings below are the recovered original files as published in [8].

`write_case.c`:

```
/*Workload B: create a test file in /tmp and write a fixed payload.*/
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

int main(void)
{
    const char *payload = "FORENSICS_TEST_2026\n";
    int fd = open("/tmp/tracer_case_study.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0)
        return 1;
    if (write(fd, payload, strlen(payload)) < 0)
        return 1;
    close(fd);
    return 0;
}
```

`connect_case.c`:

```
/*Workload C: attempt a TCP connection to a closed loopback port.*/
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>

int main(void)
{
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9);                /*discard port, expected closed*/
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);  /*127.0.0.1*/

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0)
        return 1;
    int rc = connect(fd, (struct sockaddr *)&addr, sizeof(addr));
    close(fd);
    return rc == 0 ? 0 : 2;
}
```

A.5 Trace file hashes (SHA-256)

The six traces, with these values in a `SHA256SUMS` file, are published in the evaluation directory cited in [8].

```
72757abe12e7aa636ab8dc65cedebfc20f9b13ce0c82d670f0de8ad8e59be184  A2.trace
9cfa6c612985c3c51f86ec64f72e3c87378c5d6e1820d94257976b2185348c56  A3.trace
c891899e51e0563c3160f8f929c8ae0f33b2bc80eba526b505045e65cc09722f  A.trace
0c165f0214050a0161d0999c2ebe1434f8f5514115ed2e259e59db59dbf2a30a  B.trace
27064046b4a4a3ff396ed8a8b171fa9b1a5c6e72393cb7b56c715a2c34ae45f9  C.trace
9eced3f0d536b391be78d73eb16b0278a980ecb17045f05e34517736b23bfcfa  D.trace
```

A.6 Counting method

Every line in each trace has the form `name(args) = result`, and no line in any trace failed that pattern, so lines equal records. A record whose entry was printed but whose process ended first (`exit_group`) is counted once and also listed as unmatched. Decoded-path records are those with a quoted string in a path argument position (`execve`, `access`, `openat`). The counts are descriptive, not detection metrics.